

Controalele din exemplul CRUD

Ce e fiecare control și ce proprietăți i-am pus.

Fișa explică **exact** controalele din `exemplu-crud`. Deschide proiectul alături.

Username	Email	Valid	Telefon	Varsta
andrei_pop	andrei.popescu@gmail....	☒	0722123456	25
elena_io	elena.ionescu@yahoo.c...	☒	0731987654	31
vlad_m	vlad.marcu@outlook.com	☐	0745112233	19
ralu_c	raluca.constantin@gmai...	☒	0756445566	42
cosmin_radu	cosmin.radu@gmail.com	☒	0767889900	28
ana_stan	anamaria.stan@yahoo.c...	☒	0721334455	22
marian_d	marian.dumitru@gmail.c...	☐	0732556677	50
AlexGamer	alex.neagu@outlook.com	☒	0743778899	16
cristina_v	cristina.vlad@gmail.com	☒	0754114477	35
stefan_b	stefan.badea@yahoo.c...	☒	0765225588	27

Datele userului

Username Email Password

Telefon Varsta ☒ Valid

Incarca

Adauga

Modifica

Sterge

Incarcati 10 utilizatori

Forma întreagă: șapte tipuri de control

Form — fereastra însăși

Forma e și ea un control: are `Text` , `Size` , `Controls` .

Proprietate	Valoarea pusă	Ce face
<code>Text</code>	<code>"Useri – exemplu CRUD"</code>	textul din bara de titlu
<code>ClientSize</code>	<code>684 x 448</code>	suprafața utilă , fără ramă și titlu
<code>StartPosition</code>	<code>CenterScreen</code>	unde apare la pornire

`ClientSize` nu e `Size` . `Size` include rama și bara de titlu; `ClientSize` e doar partea în care pui controale. De aia fereastra reală iese mai mare cu câțiva pixeli în lățime și cu vreo 39 în înălțime.

Label

Username

lblUsername — etichetă fixă, pusă din Properties

Incarcati 10 useri

lblStare — aceeași clasă, text schimbat din cod

Proprietate	Valoarea	Ce face
Text	"Username"	ce scrie pe ecran
AutoSize	true	se redimensionează singur după text

Când `AutoSize` e `true`, **Size e ignorat**. De aia în Designer scrie `lblStare.Size = 0, 13` — zero lățime, fiindcă `Text` e gol; când scrii ceva în el, se lățește singur.

Un singur label se schimbă din cod:

```
lblStare.Text = "Adaugat: " + u.Username;
```

Restul sunt etichete pentru casete. Niciunul n-are `Click` legat: sunt vitrină, nu butoane.

TextBox

andrei_pop

`txtUsername` — Text-ul e mereu `string`

••••••••

`txtPassword` — același control, cu `PasswordChar`

Proprietate / metodă	Unde	Ce face
<code>Text</code>	peste tot	conținutul, mereu <code>string</code>
<code>PasswordChar</code>	<code>txtPassword</code>	ce caracter se afișează în loc de litere
<code>Clear()</code>	<code>GolesteCampurile()</code>	golește caseta

`PasswordChar` ascunde pe ecran, nu în memorie

Pe ecran apar stelute, dar `txtPassword.Text` întoarce parola **în clar**. E o proprietate de afișare, nu de securitate. Ce se vede pe ecran și ce e în memorie sunt două lucruri diferite. De-aia am ascuns și coloana `Password` din tabel.

Din casetă nu iese niciodată un număr

`txtVarsta.Text` e `string`, chiar dacă s-a tastat `25`. Conversia o faci tu, și poate eșua:

```
int varsta;
if (!int.TryParse(txtVarsta.Text, out varsta))
{
    MessageBox.Show("Varsta trebuie sa fie un numar intreg.");
    return null;
}
```

`int.TryParse` nu aruncă excepție: întoarce `true` / `false`. `int.Parse` ar fi aruncat `FormatException` la primul `abc` și aplicația s-ar fi închis brusc.

Invers, ca să pui un număr în casetă: `txtVarsta.Text = u.Varsta.ToString();`

CheckBox



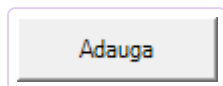
chkValid — valoarea stă în *Checked*, nu în *Text*

Proprietate	Tip	Ce face
Checked	bool	bifat sau nu
Text	string	eticheta de lângă bifă

```
u.Valid = chkValid.Checked;    // din control in obiect
chkValid.Checked = u.Valid;    // din obiect in control
```

Singurul control din formă care ține direct un `bool`, fără conversie — tipul controlului se potrivește deja cu tipul câmpului.

Button



`btnAdauga` — Name pentru tine, Text pentru utilizator

Proprietate / eveniment	Valoarea	Ce face
<code>Text</code>	<code>"Adauga"</code>	ce citește utilizatorul
<code>Name</code>	<code>btnAdauga</code>	cum îi zici tu în cod
<code>Click</code>	<code>btnAdauga_Click</code>	metoda care rulează la apăsare
<code>UseVisualStyleBackColor</code>	<code>true</code>	se desenează cu tema Windows

Legătura buton–metodă e un singur rând, scris de Designer:

```
this.btnAdauga.Click += new System.EventHandler(this.btnAdauga_Click);
```

`+=` nu e atribuire, e **abonare**. Butonul ține o listă de metode de chemat. De aia, dacă linia apare de două ori, metoda ta rulează **de două ori** la o singură apăsare — acolo te uiți când ceva se face dublu fără motiv.

`UseVisualStyleBackColor = true` înseamnă că `BackColor` e ignorat până îl pui pe `false`.

GroupBox

Datele userului

Username

andrei_pop

Email

andrei.popescu@gmail.com

Password

Telefon

0722123456

Varsta

25

☒ Valid

`grpDate` — o ramă cu titlu, care conține alte controale

Proprietate	Valoarea	Ce face
Text	"Datele userului"	eticheta din colțul ramei
TabStop	false	rama însăși nu primește focus la Tab

Important nu e rama, ci că e un **container**:

```
this.grpDate.Controls.Add(this.txtUsername);
```

`txtUsername` intră în `grpDate.Controls`, nu în `this.Controls`. Consecințe:

- **Coordonatele sunt relative la GroupBox.** `Location = 80, 25` înseamnă 80 px de la marginea ramei. Muți rama — se mută tot ce e înăuntru.
- `TabIndex` -urile se numără **în interiorul** containerului.
- `grpDate.Visible = false` ascunde tot ce e în el.

Regula: **un control aparține unui singur container, iar coordonatele lui sunt față de containerul ăla.**

DataGridView

Username	Email	Valid	Telefon	Varsta
andrei_pop	andrei.popescu@gmail....	☒	0722123456	25
elena_io	elena.ionescu@yahoo.c...	☒	0731987654	31
vlad_m	vlad.marcu@outlook.com	☐	0745112233	19
ralu_c	raluca.constantin@gmai...	☒	0756445566	42
cosmin_radu	cosmin.radu@gmail.com	☒	0767889900	28
ana_stan	anamaria.stan@yahoo.c...	☒	0721334455	22
marian_d	marian.dumitru@gmail.c...	☐	0732556677	50
AlexGamer	alex.neagu@outlook.com	☒	0743778899	16
cristina_v	cristina.vlad@gmail.com	☒	0754114477	35
stefan_b	stefan.badea@yahoo.c...	☒	0765225588	27

dgvUseri — coloanele sunt generate din datele legate

Proprietate	Valoarea	Ce face
ReadOnly	true	celulele nu se editează în tabel
AllowUserToAddRows	false	scoate rândul gol de la final
AllowUserToDeleteRows	false	Delete pe un rând nu șterge nimic
RowHeadersVisible	false	scoate coloana gri din stânga
SelectionMode	FullRowSelect	click pe o celulă selectează tot rândul
MultiSelect	false	un singur rând selectat
AutoSizeColumnsMode	Fill	coloanele împart lățimea

Primele patru spun același lucru: **tabelul e vitrină, nu formular**. Cu `ReadOnly = false`, utilizatorul ar fi editat **copia** afișată, nu lista ta, iar modificările ar fi dispărut la următorul `Incarca`.

Tabelul afișează proprietăți, nu câmpuri

Clasa	Ce iese în tabel
<code>class User { public string username; }</code>	nimic — tabel gol
<code>class User { public string Username { get; set; } }</code>	o coloană <code>Username</code>

Fără eroare, fără avertisment. Doar un tabel gol. Aceeași regulă la `ComboBox.DisplayMember` și `ListBox.DisplayMember`. **Antetul coloanei e numele proprietății** — de aia le-am numit `Telefon` și `Varsta`, nu `phoneNr` și `age`.

Cum schimbi antetul coloanelor

Implicit, antetul e numele proprietății. Ca să scrie altceva, ai **două căi** — și trebuie să alegi una, nu să le amesteci.

 **Întâi:** `AutoGenerateColumns` nu apare în fereastra Properties

Dacă ai căutat-o în lista de proprietăți a lui `dgvUser1`, între `AutoSizeColumnsMode` și `BackgroundColor` — nu e acolo. Proprietatea **există**, dar e marcată `[Browsable(false)]`, adică ascunsă de fereastra Properties. **Se setează doar din cod.** Nu e o scăpare de-a ta și nu o vei găsi derulând mai mult.

Calea A — lași coloanele generate și le redenumеști după binding

```
dgvUser1.DataSource = new List<User>(useri);  
dgvUser1.Columns["Judet"].HeaderText = "Judetul";  
dgvUser1.Columns["Populatie"].HeaderText = "Numar de locuitori";
```

Nume	Judetul	Numar de locuitori
Cluj-Napoca	Cluj	286598
Timisoara	Timis	250849
Iasi	Iasi	271692

Calea A: coloane generate automat, două antete redenumite

După binding, obligatoriu. Înainte, `Columns` e gol — coloanele nu există încă. Aceeași regulă ca la ascunderea coloanei `Password`.

`Name` rămâne numele proprietății; se schimbă doar `HeaderText`. Deci continui să indexezi cu `Columns["Populatie"]`, nu cu `Columns["Numar de locuitori"]`.

Calea B — îți definești tu coloanele

```
dgvUseri.AutoGenerateColumns = false;

DataGridViewTextBoxColumn col = new DataGridViewTextBoxColumn();
col.HeaderText = "Numar de locuitori";
col.DataPropertyName = "Populatie";
col.Name = "Populatie";
dgvUseri.Columns.Add(col);

dgvUseri.DataSource = lista;
```

Orasul	Judetul	Numar de locuitori
Cluj-Napoca	Cluj	286598
Timisoara	Timis	250849
Iasi	Iasi	271692

Calea B: coloane definite de tine, cu `HeaderText` propriu

`DataPropertyName` e legătura cu proprietatea din clasă, `HeaderText` e ce se vede pe ecran. Sunt două lucruri diferite — de aia poți scrie în antet orice text, cu spații și diacritice.

Din Designer, proprietatea `Columns` (**Collection**) deschide fereastra *Edit Columns*, unde adaugi coloanele și le scrii `HeaderText`. Dar e degeaba dacă n-ai pus `AutoGenerateColumns` pe `false` **din cod**: la binding se generează altele peste ale tale.

👉 **Pe Calea B, `Name` nu se completează singur.** Dacă uiți `col.Name`, `Columns["Populatie"]` nu mai găsește nimic — verificat: `Columns.Contains("Populatie")` întoarce `false`. Iar linia din exemplu care ascunde parola, `Columns["Password"].Visible = false`, funcționează **doar** pentru că acolo coloanele sunt generate automat și își iau numele din proprietăți.

Dacă le amesteci

Verificat: o coloană adăugată de tine, cu `AutoGenerateColumns` lăsat pe `true`, dă `Orasul | Judet | Populatie` — a ta e păstrată, restul se generează. Nu se dublează nimic, dar alege o cale și ține-te de ea.

De ce se reîncarcă tot tabelul

```
dgvUseri.DataSource = null;
dgvUseri.DataSource = new List<User>(useri);
if (dgvUseri.Columns.Contains("Password"))
{
    dgvUseri.Columns["Password"].Visible = false;
}
```

1. **null înainte** — o `List<T>` nu anunță pe nimeni când o modifici. Faci `Add` și tabelul nu află. Îl forțezi să se uite din nou.
2. **Listă nouă, nu aceeași** — `new List<User>(useri)` copiază lista, nu userii.
3. **Ascunderea vine după binding** — înainte, `Columns` e gol: coloanele nu există încă.

De la rând la obiect

```
if (dgvUseri.CurrentRow == null) return -1;
int i = dgvUseri.CurrentRow.Index;
if (i < 0 || i >= useri.Count) return -1;
```

Tabelul nu-ți dă un `User`, îți dă un **rând**. Legătura cu lista e **poziția**. Cele două verificări nu-s formalitate: `CurrentRow` e `null` când tabelul e gol, iar `SelectionChanged` se declanșează **și** în timpul reîncărcării, când tabelul și lista pot fi o clipă desincronizate. Fără ele, `useri[i]` aruncă `ArgumentOutOfRangeException`.

`SelectionChanged`, nu `CellClick`: se declanșează și la mutarea cu săgețile.

MessageBox — nu e control

Nu se trage din Toolbox și nu apare în `Designer.cs`. E o clasă cu metode statice:

```
MessageBox.Show("Varsta trebuie sa fie un numar intreg.");
```

E **modală**: linia următoare nu se execută până nu apeși OK — aceeași diferență `Show` / `ShowDialog` de la tema cu ferestre. Apare doar pe căile de refuz: un mesaj se arată când utilizatorul trebuie să afle **de ce nu s-a întâmplat** ce aștepta.

Recapitulare

Control	Citești din el	Scii în el
<code>TextBox</code>	<code>Text</code> (mereu <code>string</code>)	<code>Text</code> , <code>Clear()</code>
<code>CheckBox</code>	<code>Checked</code> (<code>bool</code>)	<code>Checked</code>
<code>Label</code>	—	<code>Text</code>
<code>Button</code>	—	<code>Text</code> ; te abonezi la <code>Click</code>
<code>GroupBox</code>	—	conține alte controale
<code>DataGridView</code>	<code>CurrentRow.Index</code>	<code>DataSource</code> , <code>Columns[...].Visible</code>

Și cele patru reguli care nu se văd din sintaxă:

1. **Un tabel legat la o listă afișează proprietăți, nu câmpuri.**
2. Din `TextBox` iese mereu `text` — conversia o faci tu, cu `TryParse` .
3. `PasswordChar` ascunde pe ecran, nu în memorie.
4. **Coordonatele unui control sunt relative la containerul lui.**

